

Modul Praktikum Flutter Lanjutan

Pertemuan 14: Hardening, Deploy dengan Nginx, dan Proyek Akhir

Durasi	Level	Bentuk kerja	Prasyarat
150 menit + proyek akhir (1–2 minggu)	Lanjut	Individu atau kelompok 2 orang	Pertemuan 8–13

1. Tujuan Pembelajaran

Setelah praktikum ini, mahasiswa mampu:

1. Membedakan konfigurasi **pengembangan** dan **produksi**, lalu menulis `docker-compose.prod.yml`.
2. Memasang **Nginx** sebagai *reverse proxy*: satu pintu masuk, pembatasan laju login (*rate limit*), dan header keamanan.
3. Menerapkan praktik keamanan dasar: rahasia lewat `.env`, validasi rahasia saat aplikasi mulai, `/docs` dimatikan, proses tanpa root, dan database tidak terekspos.
4. Mencadangkan dan memulihkan database beserta berkas unggahan.
5. Menjalankan pengujian otomatis di CI (GitHub Actions) dan uji *end-to-end* di emulator.
6. Membangun aplikasi Flutter rilis yang mengarah ke server produksi.

2. Alat dan Bahan

- Hasil pertemuan 13 (atau snapshot `pert13/tugas13`).
- Image `nginx:1.29-alpine`.
- Opsional: akun GitHub (CI) dan server/VPS atau laptop lain di jaringan yang sama sebagai server uji.

3. Teori Singkat

Aspek	Pengembangan (<code>docker-compose.yml</code>)	Produksi (<code>docker-compose.prod.yml</code>)
Port terbuka	8000, 8080, 8081, 3307	Hanya 80 (Nginx)
Kode	<i>Bind mount</i> , auto-reload	Disalin ke image, tidak berubah
API	1 proses <code>--reload</code>	Beberapa <i>worker</i> <code>uvicorn</code>
Dokumentasi <code>/docs</code>	Aktif	Mati
Laravel	<code>APP_DEBUG=true</code>	<code>APP_DEBUG=false</code> , cache konfigurasi
Paket uji (pytest, phpunit)	Terpasang	Tidak dipasang
User proses API	root	<code>app</code> (uid 1000)
Adminer	Ada	Tidak ada

Reverse proxy. Pengguna hanya berbicara dengan Nginx, lalu Nginx meneruskan request ke layanan internal berdasarkan path:

HP / Browser --:80-->	+----- docker network -----+
	Nginx -- /api/... --> api:8000 (FastAPI)
	-- /, /admin --> admin:8080 (Laravel+Filament)
	\--> mysql:3306 (tidak terbuka)
	+-----+

Keuntungannya: satu domain dan satu sertifikat HTTPS, layanan internal tidak terekspos, serta satu tempat untuk *rate limit*, batas ukuran unggahan, dan header keamanan.

****Header X-Forwarded-*.**** Di belakang proxy, aplikasi melihat Nginx sebagai klien. Nginx mengirim alamat asli lewat header X-Forwarded-For, -Proto, dan -Host. Aplikasi harus **mempercayai** header ini: `uvicorn` memakai `--proxy-headers`, sedangkan `Laravel` memakai `trustProxies`. Bila tidak, URL aset dan *redirect* akan salah.

4. Langkah Praktikum

Bagian A: Konfigurasi FastAPI untuk Produksi

1. `config.py` mendapat `app_env`, `cors_origins`, dan validasi yang **menolak berjalan** bila rahasia JWT lemah. Gagal di awal lebih baik daripada berjalan dengan kunci yang bocor:

```
JWT_SECRET_BAWAAN = "kunci-pengembangan-jangan-dipakai-di-produksi"

class Settings(BaseSettings):
    # "development" atau "production".
    app_env: str = "development"
    ...
    jwt_secret: str = JWT_SECRET_BAWAAN
    ...
    # Asal (origin) yang boleh memanggil API dari browser, dipisah koma.
    # Aplikasi Android/iOS tidak terkena CORS; ini untuk Flutter Web.
    cors_origins: str = ""

    @property
    def produksi(self) -> bool:
        return self.app_env == "production"

    @model_validator(mode="after")
    def cek_produksi(self):
        if self.produksi and (self.jwt_secret == JWT_SECRET_BAWAAN or len(self.jwt_secret) < 32):
            raise ValueError("JWT_SECRET wajib diganti (minimal 32 karakter) di produksi")
        return self
```

2. `main.py` mematikan dokumentasi di produksi dan memasang CORS hanya bila diatur:

```
app = FastAPI(
    title="TabungKu API",
    version="1.0.0",
    docs_url=None if settings.produksi else "/docs",
    redoc_url=None,
    openapi_url=None if settings.produksi else "/openapi.json",
)

if settings.cors_origins:
    app.add_middleware(
        CORSMiddleware,
        allow_origins=[o.strip() for o in settings.cors_origins.split(",")],
        allow_methods=["*"],
        allow_headers=["Authorization", "Content-Type"],
    )
```

3. `start.sh` memilih mode berdasarkan `APP_ENV`:

```
if [ "$APP_ENV" = "production" ]; then
    echo "Menjalankan API (produksi, ${API_WORKERS:-2} worker)..."
    # --proxy-headers: percayai X-Forwarded-* dari Nginx.
    # --root-path /api: API diakses lewat Nginx di bawah awalan /api.
    exec uvicorn app.main:app --host 0.0.0.0 --port 8000 \
        --workers "${API_WORKERS:-2}" --proxy-headers --forwarded-allow-ips="*" \
        --root-path /api
fi
```

```
echo "Menjalankan API (pengembangan, auto-reload)..."
exec uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

4. Dockerfile menerima ARG INSTALL_DEV (produksi: hanya requirements.txt) dan membuat user biasa:

```
ARG INSTALL_DEV=true

COPY requirements.txt requirements-dev.txt ./
RUN if [ "$INSTALL_DEV" = "true" ]; then \
    pip install --no-cache-dir -r requirements-dev.txt; \
else \
    pip install --no-cache-dir -r requirements.txt; \
fi

COPY . .

# User biasa (bukan root) untuk produksi; dipilih lewat `user: app` di
# docker-compose.prod.yml.
RUN useradd --create-home --uid 1000 app \
    && mkdir -p /data/uploads && chown app:app /data/uploads
```

5. Tambahkan tests/test_produksi.py yang memastikan Settings (app_env="production", jwt_secret=JWT_SECRET_BAWAAN) gagal dan rahasia 40 karakter diterima.

Bagian B: Laravel di Belakang Proxy

1. bootstrap/app.php:

```
->withMiddleware(function (Middleware $middleware): void {
    // Di produksi Laravel berada di belakang Nginx. Percayai header
    // X-Forwarded-* agar URL, skema, dan IP klien terbaca benar.
    $middleware->trustProxies(at: '*');
})
```

2. Dockerfile admin: tambahkan ARG COMPOSER_NO_DEV=0 dan ENV COMPOSER_NO_DEV=\${COMPOSER_NO_DEV} sebelum composer install, sehingga produksi tidak memasang paket pengujian.
3. docker/start.sh: setelah filament:upgrade, tambahkan cache untuk produksi:

```
if [ "$APP_ENV" = "production" ]; then
    # Cache konfigurasi, rute, view, dan komponen Filament agar lebih cepat.
    php artisan optimize
    php artisan filament:optimize
fi
```

Bagian C: Nginx

nginx/default.conf:

```
# Batasi percobaan login: 5 per menit per alamat IP (mencegah tebak password).
limit_req_zone $binary_remote_addr zone=login:10m rate=5r/m;
limit_req_status 429;

server {
    listen 80;
    server_name _;

    # Sedikit di atas batas foto 2 MB agar pesan galat datang dari FastAPI.
    client_max_body_size 3m;

    add_header X-Content-Type-Options nosniff always;
    add_header X-Frame-Options SAMEORIGIN always;
    add_header Referrer-Policy strict-origin-when-cross-origin always;
    server_tokens off;
```

```

# $http_host membawa port (misalnya localhost:8088); $host tidak.
# Laravel memakainya untuk menyusun URL aset CSS/JS Filament.
proxy_set_header Host $http_host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header X-Forwarded-Host $http_host;

location = /api/auth/login {
    limit_req zone=login burst=3 nodelay;
    proxy_pass http://api:8000/auth/login;
}

# Garis miring di akhir proxy_pass membuang awalan /api.
location /api/ {
    proxy_pass http://api:8000/;
}

location / {
    proxy_pass http://admin:8080;
}
}

```

Pelajaran dari pengujian. Versi awal memakai \$host, yang tidak membawa nomor port. Saat diuji di port 8088, halaman login Filament memuat CSS/JS dari `http://127.0.0.1/...` (tanpa :8088), sehingga gagal dan tombol login tidak berfungsi. Kesalahan seperti ini hanya terlihat bila diuji dengan **browser sungguhan**, tidak cukup dengan `curl`.

Bagian D: docker-compose.prod.yml

Berkas terpisah (bukan *override*) agar mudah dibaca. Potongan pentingnya:

```

name: tabungku-prod

services:
  mysql:
    image: mysql:8.4
    restart: always
    # ... sama seperti pengembangan, TANPA "ports"

  api:
    build:
      context: ./backend
      args:
        INSTALL_DEV: "false"
    restart: always
    user: app
    environment:
      APP_ENV: production
      API_WORKERS: ${API_WORKERS:-2}
      DATABASE_URL: mysql+pymysql://${MYSQL_USER}:${MYSQL_PASSWORD}@mysql:3306/${MYSQL_DATABASE}
      JWT_SECRET: ${JWT_SECRET}
      CORS_ORIGINS: ${CORS_ORIGINS:-}
    volumes:
      - uploads:/data/uploads # TANPA bind mount kode
      # healthcheck dan depends_on seperti pengembangan

  admin:
    build:
      context: ./admin
      args:
        COMPOSER_NO_DEV: "1"
    restart: always
    environment:
      APP_ENV: production
      APP_DEBUG: "false"
      APP_URL: ${PUBLIC_URL}
      API_PUBLIC_URL: ${PUBLIC_URL}/api
      LOG_LEVEL: warning

```

```
# ... DB_*, SESSION_DRIVER, ADMIN_* seperti pengembangan

nginx:
  image: nginx:1.29-alpine
  restart: always
  ports:
    - "${HTTP_PORT:-80}:80"
  volumes:
    - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
  depends_on:
    api:
      condition: service_healthy
    admin:
      condition: service_healthy
```

Tambahkan ke `.env.example`: `PUBLIC_URL=http://localhost` (alamat publik tanpa garis miring akhir), `HTTP_PORT=80`, `API_WORKERS=2`, dan `CORS_ORIGINS=`. Tambahkan `backups/` ke `.gitignore`.

Jalankan (hentikan dulu stack pengembangan):

```
docker compose down
cp .env.example .env # GANTI semua password, JWT_SECRET, ADMIN_APP_KEY, ADMIN_PASSWORD
docker compose -f docker-compose.prod.yml up -d --build
docker compose -f docker-compose.prod.yml ps
```

Bagian E: Daftar Periksa Keamanan

Isi tabel ini di laporan (snapshot sudah diuji dengan hasil seperti kolom kanan):

No	Uji	Hasil yang diharapkan
1	<code>curl http://SERVER/api/health</code>	200, database: ok
2	<code>curl http://SERVER/api/docs</code>	404
3	Buka <code>http://SERVER/</code> di browser	Diarahkan ke <code>/admin</code> ; login berhasil; CSS tampil
4	<code>curl -I http://SERVER/admin/login</code>	Ada <code>X-Content-Type-Options</code> , <code>X-Frame-Options</code> ; <code>Server: nginx</code> tanpa versi
5	Login salah 6 kali berturut-turut	Permintaan ke-5 dan seterusnya dibalas 429
6	Hubungkan ke <code>SERVER:3306</code> atau <code>:3307</code>	Ditolak (port tidak terbuka)
7	<code>docker compose -f docker-compose.prod.yml exec api id</code>	<code>uid=1000 (app)</code>
8	<code>... exec admin php artisan about --only=environment</code>	Environment <code>production</code> , Debug Mode <code>OFF</code>
9	Unggah foto 2,5 MB lewat <code>/api</code>	413 dari FastAPI ("Ukuran foto maksimal 2 MB")
10	Unggah berkas 4 MB	413 dari Nginx (ditolak sebelum mencapai API)
11	Ubah <code>JWT_SECRET</code> di <code>.env</code> menjadi pendek, lalu <code>up -d api</code>	Container API gagal dan log menampilkan pesan validasi

Aktifkan kembali rahasia yang benar setelah uji no. 11. Untuk mengulang uji no. 5 tanpa menunggu satu menit, jalankan `docker compose -f docker-compose.prod.yml restart nginx`.

Bagian F: Backup dan Restore

`scripts/backup.sh` menyimpan dump database dan arsip foto ke `backups/`:

```
#!/bin/sh
set -e
COMPOSE="docker compose -f ${1:-docker-compose.yml}"
WAKTU=$(date +%Y%m%d-%H%M%S)
mkdir -p backups

# --single-transaction: salinan konsisten tanpa mengunci tabel InnoDB.
$COMPOSE exec -T mysql sh -c 'exec mysqldump --single-transaction \
    --no-tablespaces -u"$MYSQL_USER" -p"$MYSQL_PASSWORD" "$MYSQL_DATABASE" \
    | gzip > "backups/db-$WAKTU.sql.gz"

$COMPOSE exec -T api tar -C /data -czf - uploads > "backups/uploads-$WAKTU.tar.gz"
```

`scripts/restore.sh WAKTU [berkas-compose]` melakukan kebalikannya (lihat snapshot). Latihan wajib: **cadangkan, hapus data, lalu pulihkan**:

```
sh scripts/backup.sh docker-compose.prod.yml
# hapus beberapa kegiatan lewat dashboard
sh scripts/restore.sh 20261006-175155 docker-compose.prod.yml
```

Backup yang tidak pernah dicoba dipulihkan bukanlah backup. Simpan salinan di luar server (misalnya penyimpanan awan) dan jadwalkan dengan `cron`.

Bagian G: CI dengan GitHub Actions

Snapshot berisi `.github/workflows/ci.yml` dengan tiga *job*: `backend` (`ruff check + pytest --cov`), `mobile` (`flutter analyze + flutter test`), dan `docker` (`build image produksi`). Salin ke `.github/workflows/` di **akar** repositori proyek Anda, lalu sesuaikan `working-directory`. Setiap *push* dan *pull request* akan diuji otomatis; tampilkan *badge* status di README.

Bagian H: Flutter Rilis

Alamat API produksi diberikan saat build. Tidak perlu mengubah kode karena `apiUrl` sudah membaca `--dart-define` sejak pertemuan 8:

```
flutter build apk --release --dart-define=API_URL=http://192.168.1.10/api
```

Foto bukti otomatis menjadi `http://192.168.1.10/api/uploads/...`, yang diteruskan Nginx ke FastAPI.

HTTPS. `usesCleartextTraffic="true"` hanya untuk jaringan lokal. Di internet publik, wajib memakai HTTPS (misalnya Nginx + Let's Encrypt/Certbot, atau layanan seperti Cloudflare Tunnel), lalu hapus izin `cleartext`.

Bagian I: Uji End-to-End (Opsional)

Snapshot `tugas14/mobile/integration_test/alur_test.dart` menjalankan aplikasi sungguhan terhadap Docker sungguhan: login, menunggu banner pengumuman, menambah pengeluaran berkategori, membuka Laporan (termasuk kartu bulanan), Pengaturan, dan Status Server, sambil menyimpan tangkapan layar:

```
docker compose up -d
docker compose exec api python -m scripts.seed_demo
cd mobile
flutter drive --driver=test_driver/integration_test.dart \
    --target=integration_test/alur_test.dart \
    --dart-define=API_URL=http://localhost:8000 # simulator iOS
# emulator Android: --dart-define=API_URL=http://10.0.2.2:8000
```

Tangkapan layar tersimpan di `mobile/screenshots/`. Setiap kali dijalankan, test ini menambah satu kegiatan pada akun uji.

5. Proyek Akhir

Kembangkan TabungKu Cloud dengan **satu fitur besar baru** yang menyentuh **ketiga lapisan** (database + API, dashboard, aplikasi), atau dua fitur sedang. Contoh lengkap tersedia di snapshot `tugas14`:

Pengumuman dari admin (tabel + migrasi 0005, `GET /pengumuman` sesuai rentang tayang, Resource Filament dengan status Tayang/Terjadwal/Berakhir, dan banner di aplikasi yang dapat ditutup).

Pilihan fitur	Lapisan yang terlibat
Target tabungan jangka panjang (misalnya "Laptop Rp 8 jt"), progres dari tabungan harian	DB + API + Flutter + grafik admin
Kegiatan berulang (uang saku mingguan otomatis)	DB + API (job terjadwal) + Flutter
Anggaran per kategori per bulan + peringatan saat terlampaui	DB + API + Flutter + widget admin
Notifikasi (FCM) saat admin mengirim pengumuman	API + Firebase + Flutter
Kas kelompok: beberapa pengguna dalam satu grup	DB (relasi banyak-ke-banyak) + API + Flutter + admin
Log audit perubahan data oleh admin	Laravel (observer) + Resource baru

Ketentuan:

- Perubahan skema **wajib** lewat revisi Alembic baru; tidak boleh mengubah revisi lama.
- Endpoint baru dilindungi JWT, dengan isolasi data bila datanya milik pengguna.
- Minimal 3 test `pytest` baru, 1 test Flutter, dan 1 feature test Filament.
- Stack produksi berjalan dengan `docker-compose.prod.yml`, dan daftar periksa bagian E lulus.
- **README** proyek berisi: arsitektur, cara menjalankan (pengembangan dan produksi), daftar endpoint, akun demo, dan hasil uji.
- **Demo 10 menit**: alur pengguna di HP, alur admin di dashboard, perubahan admin yang terlihat di HP, `pytest`, dan backup/restore.

6. Rubrik Penilaian

Komponen	Bobot
Compose produksi + Nginx + daftar periksa keamanan (A–E)	20%
Backup dan restore terbukti (F)	10%
CI berjalan, atau bukti pengujian otomatis setara (G)	5%
Proyek akhir: fungsionalitas fitur di tiga lapisan	30%
Proyek akhir: kualitas kode, migrasi, keamanan endpoint	15%
Proyek akhir: pengujian	10%
Dokumentasi dan demo	10%

7. Pertanyaan Refleksi

1. Mengapa MySQL tidak boleh membuka port ke internet, padahal dashboard dan API menggunakannya?
2. Rate limit 5 per menit di Nginx diterapkan per IP. Apa kelemahannya bila banyak mahasiswa memakai satu jaringan kampus (satu IP publik)?
3. Apa yang terjadi bila server mati total dan backup hanya disimpan di server yang sama?
4. Sebutkan tiga keputusan desain di pertemuan 8–14 yang memudahkan deploy. Contoh: alamat API dari `--dart-define`, path relatif untuk foto.
5. Bila jumlah pengguna naik seratus kali lipat, bagian mana yang pertama kali menjadi hambatan?

8. Troubleshooting Umum

Masalah	Solusi
Dashboard produksi tanpa CSS atau tombol login tidak bereaksi	Periksa URL aset di sumber halaman. Bila port hilang, Nginx harus memakai <code>Host \$http_host</code> dan Laravel <code>trustProxies</code>
<code>/api/docs</code> masih bisa dibuka	<code>APP_ENV=production</code> belum sampai ke container API. Periksa bagian <code>environment</code>
<code>dependency failed to start: container ... is unhealthy</code> saat volume baru	MySQL belum benar-benar siap. Healthcheck harus memakai <code>-h 127.0.0.1</code> (TCP). Jalankan ulang <code>up -d</code>
502 Bad Gateway sesaat setelah <code>up</code>	API/admin masih menyala. Tunggu sampai <code>ps</code> menunjukkan <i>healthy</i>
<code>PermissionError</code> unggah di produksi	Volume <code>uploads</code> lama dibuat oleh root. Gunakan volume baru atau <code>chown</code> lewat container root sekali
Login admin produksi ditolak: "These credentials do not match"	Admin dibuat dari <code>ADMIN_EMAIL/ADMIN_PASSWORD</code> saat pertama kali. Mengubah <code>.env</code> setelahnya tidak mengganti password admin yang sudah ada
Aplikasi rilis tidak tersambung	Periksa <code>--dart-define=API_URL=.../api</code> (dengan <code>/api</code>) dan izin <i>cleartext</i> atau HTTPS
<code>restore.sh</code> gagal Access denied	Variabel <code>MYSQL_*</code> di <code>.env</code> harus sama dengan saat backup dibuat

9. Referensi

- Docker Compose di produksi: docs.docker.com/compose/how-tos/production
- Nginx `limit_req` dan `proxy_pass`: nginx.org/en/docs
- FastAPI, *Behind a Proxy* dan *Deployment*: fastapi.tiangolo.com/advanced/behind-a-proxy
- Laravel, *Trusting Proxies*: laravel.com/docs/requests#configuring-trusted-proxies
- OWASP Top 10: owasp.org/Top10
- GitHub Actions: docs.github.com/actions; flutter-action: github.com/subosito/flutter-action
- Flutter integration test: docs.flutter.dev/testing/integration-tests